Test Strategy — peviitor.ro

> ***Project***: *peviitor.ro — Locul de muncă visat, la un click distanță*
> ***Author***: *Diana Dragoi*
> ***Date***: *June 2026*
> ***Version***: *1.0*

Document Approval

Role	Name	Signature	Date
Reviewed By			
Project Manager	Diana Dragoi	_____	//2026
QA Lead	Ana-Maria Talmacel	_____	//2026
Approved By			
Product Owner	Boga Sebastian-Nicolae	_____	//2026

---

1. SCOPE

1.1 Purpose of the Document

This document defines the test strategy for the **peviitor.ro** platform — the open source job search engine for Romania. A core strategic principle is that the platform is built exclusively with **open source and free technologies**, and this commitment is not subject to change. Its purpose is to establish a unified testing approach covering all major platform components, from front-end to the indexing infrastructure, with the following scope objectives for each area:

> ⚠️ ***Always keep in mind***: *The primary mission of peviitor.ro is to provide users with a* ***comprehensive, up-to-date aggregated database of all jobs in Romania***. *Every test, decision, and validation should serve this goal — ensuring job seekers can find relevant opportunities quickly and reliably.*

Apache SOLR
- Upgrade to the latest stable version
- Maintain two dedicated cores: `job` and `company`
- Ensure the SOLR schema matches the Job and Company models defined in peviitor_core
- Guarantee data consistency — no orphaned fields, no missing mandatory fields from the job or company model
- Implement Basic Authentication as a security layer

PHP API (api.peviitor.ro)
- Maintain two versions: **v0** for development and testing, **v1** for production
- Keep the codebase simple, fast, and easy to maintain
- Ensure the code is easily deployable
- Ensure the code is testable
- Eliminate security vulnerabilities
- Secure all API endpoints

UI (search-engine)
- Keep the UI simple to write and maintain
- Ensure the UI conforms to the UX Design (Figma)
- Host the UI on GitHub Pages
- Ensure the UI remains accessible even if the BFF PHP API is down (graceful degradation)

Search Functionality
- Make the search interface easy to use
- Deliver relevant search results
- Provide useful filters for users (location, company, tags, work mode)

1.2 Out of Scope

- Individual scraper testing (each scraper is handled separately by the scraping teams)
- Validator (admin.peviitor.ro) — will be covered in a separate test strategy document
- Mobile application testing (androidAPP) — will be covered in a separate test strategy for mobile testing
- orase.peviitor.ro — will be covered in a separate test strategy document
- firme.zira.ro and admin.zira.ro — will be covered in a separate test strategy document

1.3 General Objectives

- Ensure the quality of aggregated data (jobs, companies)
- Validate search engine functionality (SOLR)
- Confirm the correctness of the REST API (api.peviitor.ro)
- Verify the user experience in the web interface (search-engine)
- Identify defects early through manual and automated testing
- Ensure availability and performance on the existing infrastructure (Raspberry Pi 5 + Pi 4, Apache, NginX Proxy Manager, CloudFlare)
- Handle errors gracefully — clear messages for no results, invalid input, unavailable external links
- Validate input handling — long input (>500 chars), special characters, empty search, case sensitivity
- Verify correct integration with external links (LinkedIn, GitHub, Discord, etc.)

1.4 Quality Benefits Targeted

- **Functionality**: All features work as specified
- **Performance**: Response times < 2s for searches, daily indexing
- **Security**: OWASP Top 10 compliance, input sanitization, API endpoint security
- **Reliability**: Correct data, no dead links, expired jobs automatically removed
- **Resilience**: Platform remains operational under stress (API down, SOLR offline)
- **Disaster Recovery**: SOLR index backup and restoration
- **Redundancy**: Failover mechanisms for critical components
- **Scalability**: Ability to aggregate 40,000+ jobs updated daily
- **Validity**: Compliance with the data schema (Job Model, Company Model)
- **Accessibility**: WCAG 2.2 Level AA compliance
- **Usability**: Intuitive search flow, clear filters, helpful error messages, mobile-friendly experience
- **Browser Compatibility**: Latest 2 versions of Chrome, Firefox, Edge, Safari
- **Mobile Responsiveness**: All pages functional on viewports ≥ 320px
- **SEO**: Valid meta tags, semantic HTML, proper heading hierarchy (h1-h6), Open Graph tags required

2. TESTING APPROACH

2.1 REQUIREMENTS

This section describes where testers can find the **functional requirements** for each component. Every component (UI, API, SOLR, scrapers, etc.) has its own dedicated GitHub repository — see Appendix B for the full list.

The **Software Architecture Design (SAD)** document at sad.peviitor.ro is the primary reference for testing — it captures the architecture, component interactions, technology decisions, and the expected behavior of the entire platform. The **Infrastructure Design Document (IDD)** at idd.peviitor.ro covers the infrastructure topology, server configuration, networking, and deployment architecture. Testers should consult SAD and IDD as the authoritative sources when writing and validating test cases.

Apache SOLR

Source	Location	What It Contains
peviitor_core README	github.com/peviitor-ro/peviitor_core	SOLR schema, field rules, indexing workflows, status flow
Documentation	sad.peviitor.ro — Software Architecture Design	SOLR architecture, core configuration, technology stack
GitHub Issues	github.com/peviitor-ro/peviitor_core	SOLR-related feature requests, bug reports, schema change discussions

PHP API (api.peviitor.ro)

Source	Location	What It Contains
Swagger UI	test.peviitor.ro/swagger-ui	API contract: endpoints, request/response schemas, status codes
Documentation	sad.peviitor.ro — Software Architecture Design	API architecture, component interactions, versioning strategy
GitHub Issues	github.com/peviitor-ro/api.peviitor.ro	API feature requests, bug reports, endpoint discussions

UI (search-engine)

Source	Location	What It Contains
Figma Designs	Figma - Pe Viitor	UI mockups, design system, component specs, user flows
GitHub Issues	github.com/peviitor-ro/search-engine	Feature requests, bug reports, acceptance criteria
GitHub Releases	github.com/peviitor-ro/search-engine/releases	Release notes with changelog, new features, fixed bugs

Search Functionality

Source	Location	What It Contains
Figma Designs	Figma - Pe Viitor	Search UI mockups, filter designs, user flow
peviitor_core README	github.com/peviitor-ro/peviitor_core	Search fields, facet configuration, relevance settings

Source	Location	What It Contains
Documentation	sad.peviitor.ro — Software Architecture Design	Search architecture, SOLR query flow, indexing pipeline
GitHub Issues	github.com/peviitor-ro/search-engine	Search-related feature requests, bug reports, acceptance criteria

General

Source	Location	What It Contains
Discord	discord.gg/KPMkdUfQNu	Real-time discussions, clarifications, decisions, meeting notes
Onboarding Portal	onboarding.peviitor.ro	General project overview, contributor guide, how-to guides
ROADMAP (tracking & planning)	github.com/orgs/peviitor-ro/projects/78	Task tracking, sprint planning, environment readiness, deploy status, team activity — the single source of truth for what is being worked on across all components

> ***Rule**: When a requirement is unclear or missing, the tester **must** ask for clarification in Discord or via a GitHub Issue before writing test cases. Never guess the expected behavior. For functional requirements, contact the **Product Owner**; for process or priority clarifications, contact the **Project Manager**. When in doubt, start with a GitHub Issue tagged `question`.*

2.2 Non-Functional REQUIREMENTS

This section describes where testers can find the **non-functional requirements** for each category. As with functional requirements, the **Software Architecture Design (SAD)** at sad.peviitor.ro and the **Infrastructure Design Document (IDD)** at idd.peviitor.ro are the primary references — SAD defines performance targets, security architecture, and other software-level concerns; IDD covers infrastructure specs, network topology, server configuration, and disaster recovery.

Performance

Source	Location	What It Contains
Documentation	sad.peviitor.ro — Software Architecture Design	Infrastructure specs (Raspberry Pi 5 / Pi 4), capacity, response time targets
GitHub Issues	github.com/peviitor-ro per repo	Performance-related bug reports, load concerns, optimization requests

Security

Source	Location	What It Contains
OWASP Top 10	owasp.org/www-project-top-ten	Industry-standard web application security risks
GitHub Security Tab	github.com/peviitor-ro — Security tab per repo	Secret Scanning alerts, Code Scanning (CodeQL) results, Dependabot vulnerability alerts
Documentation	sad.peviitor.ro — Software Architecture Design	Authentication setup, CORS configuration, security architecture
peviitor_core	github.com/peviitor-ro/peviitor_core	SOLR basic auth scripts, security-related shell scripts

Accessibility

Source	Location	What It Contains
WCAG 2.2	w3.org/TR/WCAG22	W3C accessibility standard — Level AA compliance criteria
Figma Designs	Figma - Pe Viitor	Color contrast specs, component accessibility, focus states
GitHub Issues	github.com/peviitor-ro/search-engine	Accessibility-related bug reports, screen reader issues

Usability

Source	Location	What It Contains
Figma Designs	Figma - Pe Viitor	UX flows, search interaction design, mobile layouts, error state designs
Discord	discord.gg/KPMkdUfQNu	Community feedback, usability complaints, suggestions
GitHub Issues	github.com/peviitor-ro/search-engine	User-reported usability issues, feature requests

Additional NFRs (Reliability, Availability, Compatibility, SEO)

Source	Location	What It Contains
Documentation	sad.peviitor.ro — Software Architecture Design	Uptime targets, backup strategy, browser support policy, SEO requirements
GitHub Issues	github.com/peviitor-ro per repo	Reliability bugs, browser compatibility reports, SEO-related issues
Onboarding Portal	onboarding.peviitor.ro	General project standards, compatibility guidelines

> ***Rule**: When a non-functional requirement is unclear or missing, contact the **Product Owner** for target values or the **Project Manager** for priority. For security-specific questions, escalate to the **Security Tester**. Never guess the expected behavior.*

2.3 TOOLS

This section lists all testing tools used across the project, split by area, so testers know what tools are available and where to access them.

Frontend Testing

Tool	What It's Used For	Where to Access
Manual exploratory testing	Ad-hoc functional & UI validation	— (tester expertise)
Chrome DevTools	Debugging, performance, network, console	Built into Chrome — `F12` / `Ctrl+Shift+I`
React DevTools	React component inspection, state debugging	Chrome extension — chromewebstore.google.com
Sentry	Error monitoring, crash reporting	sentry.io — account required
Jest	JavaScript unit tests (search-engine)	jestjs.io — `npm install jest`
Vitest	JavaScript unit tests (alternative)	vitest.dev — `npm install vitest`
Playwright	Automated E2E testing	playwright.dev — `npm init playwright`

API Testing

Tool	What It's Used For	Where to Access
Postman	API endpoint testing, collections	postman.com/downloads — free tier
Bruno	API endpoint testing (open source)	usebruno.com/downloads — open source
REST Client	Quick API testing in editor	VS Code extension — marketplace.visualstudio.com
Apache JMeter	Functional API testing, parameterized requests, assertions	jmeter.apache.org — free
PHPUnit	Automated PHP unit & integration tests	docs.phpunit.de — install via Composer
Swagger UI	Interactive API documentation	test.peviitor.ro/swagger-ui — public

SOLR Indexing Testing

Tool	What It's Used For	Where to Access
SOLR Admin UI	Schema management, querying, index inspection	Test: testsolr.peviitor.ro / Prod: solr.peviitor.ro
curl	Command-line SOLR queries & API calls	Built into Windows/macOS/Linux
PHP scripts (peviitor_core)	Indexing workflows, schema validation	github.com/peviitor-ro/peviitor_core

Data Validation Testing

Tool	What It's Used For	Where to Access
admin.peviitor.ro	Manual job validation, status flow	admin.peviitor.ro
peviitor_core/tests	Automated data validation scripts	github.com/peviitor-ro/peviitor_core/tests

Cross-Component Integration

Tool	What It's Used For	Where to Access
Manual end-to-end tests	Full flow validation across components	— (tester expertise)
Postman collections	Integration API tests across endpoints	postman.com/downloads
Integration test scripts	Automated component interaction tests	github.com/peviitor-ro per repo

Test Management

Tool	What It's Used For	Where to Access
GitHub	Test case writing, bug tracking, project management, CI/CD via GitHub Actions	github.com/peviitor-ro
TestLink	Test case management, test plans, test execution tracking, reports	testlink.org — self-hosted or hosted instance
ROADMAP project	Sprint planning, task tracking, environment readiness, deploy status, team activity	github.com/orgs/peviitor-ro/projects/78

Non-Functional Testing

Tool	What It's Used For	Where to Access
Lighthouse	Performance & accessibility audits	Built into Chrome DevTools or CLI: `npm install -g lighthouse`
Apache JMeter	Performance / load testing, API & SOLR query performance	jmeter.apache.org — free
k6	Performance / load testing	grafana.com/docs/k6
OWASP ZAP	Security testing	zaproxy.org/download
axe DevTools	WCAG accessibility audit	Chrome extension — chromewebstore.google.com
WAVE	Accessibility evaluation	Chrome extension — chromewebstore.google.com
GitHub Security Tab	Secret Scanning, Code Scanning, Dependabot alerts	github.com/peviitor-ro — Security tab per repo
BrowserStack	Cross-browser testing on real devices & browsers	browserstack.com
Chrome DevTools device emulation	Mobile viewport testing, responsive design checks	Built into Chrome — `F12` / `Ctrl+Shift+I`

Communication

Tool	What It's Used For	Where to Access
Discord	Team communication, bug discussions, clarifications, deploy notifications	discord.gg/KPMkdUfQNu — public invite

Design

Tool	What It's Used For	Where to Access
Figma	UI design review, usability validation, component specs	figma.com — Pe Viitor design

Version Control

Tool	What It's Used For	Where to Access
GitHub Desktop	Git GUI for version control, commit management, branch handling	desktop.github.com — free

Local Development & Infrastructure

Tool	What It's Used For	Where to Access
Docker Desktop	Local environment setup (SOLR, API), container management	docker.com/products/docker-desktop — free tier
CloudFlare	CDN, DNS, DDoS protection, SSL, performance analytics	dash.cloudflare.com

Development Tools

Tool	What It's Used For	Where to Access
Visual Studio Code	Code editor, testing scripts, REST Client extensions, debugging	code.visualstudio.com — free
opencode	AI-powered coding assistant for development & testing tasks	opencode.ai — CLI tool

2.4 TECHNOLOGIES

This section describes the key technologies used across the platform, split by component.

Frontend (search-engine)

Technology	Purpose
React 18	JavaScript library for building the user interface
Redux Toolkit	State management for search, filters, and user interactions
Vite 5	Build tool and dev server
Tailwind CSS 3.4	Utility-first CSS framework for responsive design
React Router v6	Client-side routing between pages
Radix UI	Accessible UI primitives (tooltips, dialogs)
Lucide React	Icon library
Sentry React	Error monitoring and performance tracking
Microsoft Clarity	User behavior analytics
GitHub Pages	Hosting and delivery of the static frontend

API (api.peviitor.ro)

Technology	Purpose
PHP	Backend language for API logic and SOLR proxying
Apache	Web server with .htaccess configuration
REST API	Architectural style for API endpoints — v0 (dev/test) and v1 (production) only
Swagger / OpenAPI	API documentation and contract definition
GitHub Actions	Automated deployment pipeline

Core Library (peviitor_core)

Technology	Purpose
PHP	Shared models, data validation, SOLR indexing, status flow logic
Shell scripts	Cron jobs, index optimization, URL validation pipeline

Search & Indexing

Technology	Purpose
Apache SOLR	Search engine — dedicated cores for `job` and `company`
Basic Authentication	Security layer for SOLR admin access

Scrapers

Technology	Purpose
Python	Primary language for web scrapers
JavaScript / Node.js	Secondary scraper language
Java	Used in based_scraper_java
Apache JMeter	JMeter-based scrapers for certain targets

Infrastructure & DevOps

Technology	Purpose
Raspberry Pi 5 (16GB RAM)	Production server — hosts NginX Proxy Manager, Apache, PHP API, and SOLR
Raspberry Pi 4	Test server — hosts Apache, PHP API, NginX Proxy Manager
Apache	Web server delivering the PHP API
NginX Proxy Manager	Reverse proxy with web UI — SSL management, request routing
CloudFlare	CDN, DNS, DDoS protection, SSL termination
Docker	Containerization for local development and SOLR instances
GitHub Actions	CI/CD — automated builds, tests, and deployments
Google Sites	Hosts certain project static sites
SonarQube	Code quality and static analysis
Sentry	Error monitoring for frontend and API

2.5 Frontend Testing (search-engine)

Area	Approach
Functional Testing	Verify search flow: input query → API call → display results; filter by location, company, tags, work mode; pagination; empty states; error states
UI / Visual Testing	Layout consistency, responsive design (mobile/tablet/desktop), Tailwind CSS classes, component rendering with React
UI / GUI Testing (Figma)	Validate UI against Figma designs — layout, alignment, colors, fonts, spacing, responsive breakpoints, interactive states (hover, focus, active). Confirm correct rendering in both portrait and landscape orientations.
Automation Testing	Automate critical E2E flows with Playwright: job search, filter application, pagination, job detail navigation. Target: 80% critical path coverage. Integrate into GitHub Actions for regression on each release.
Regression Testing	Run on each release (~weekly), verify critical paths (search, filter, company page, job detail)
Backward Compatibility	Test against previous front-end versions (v01.peviitor.ro, v02.peviitor.ro) to ensure they continue to work with the latest API. The PHP API must remain backward compatible — all front-end versions (current and previous) must function correctly regardless of API evolution. Source code: **v01** → peviitor-ro/v01; **v02** → repo creation tracked in search-engine#1101.

2.6 API Testing (api.peviitor.ro)

Area	Approach
Endpoint Validation	Verify REST endpoints — **v0** (development/testing) and **v1** (production) are the only maintained versions. All functionality from v2–v7 must be migrated into v0 and v1; intermediate versions (v2–v7) will be deleted once migration is complete and must not exist in the future.
Query Parameters	Test `q`, `location`, `company`, `tags`, `workmode`, `page`, `limit` — valid, invalid, empty, boundary
Error Handling	400 Bad Request, 404 Not Found, 500 Server Error — proper error messages and HTTP codes
SOLR Proxy Behavior	Verify API correctly proxies requests to SOLR; handle SOLR timeout/offline gracefully

Area	Approach
Backward Compatibility	The PHP API must support all front-end versions (current, v01, v02). Verify that v0 and v1 endpoints return correct responses for requests from older front-end clients.

2.7 SOLR Indexing Testing

Area	Approach
Schema Compliance	Validate Job Model fields (url, title, company, location, tags, workmode, salary, etc.) against SOLR schema
Diacritics Search	Romanian diacritics: search "București" → matches "Bucuresti" and vice versa
Full-text Search	Verify relevance scoring, partial matches, phrase matching
Facet Search	Verify faceted counts by location, company, workmode, tags
CRUD Operations	Index, update, delete documents; verify atomic updates, expiration cleanup (daily @ 02:00)
URL Validation Pipeline	Verify daily @ 06:00: HEAD requests, 404 deletion, expired content detection

2.8 Data Validation Testing

Area	Approach
Job Status Flow	Verify `scraped` → `tested` / `verified` → `published` transitions
Field Validation	Required fields (url, title), formats (url, date ISO8601, salary format), max length (title ≤ 200 chars)
Company Matching	`company` field must match Company.name (case insensitive, diacritics accepted)
Expiration Logic	`expirationdate` = vdate + 30 days max; expired jobs deleted automatically
Duplicate Detection	`url` must be unique; reject duplicate entries

2.9 Cross-Component Integration

Area	Approach
Frontend ↔ API	Verify search results match API responses, filters translate to correct query params, pagination works end-to-end
API ↔ SOLR	Verify API correctly queries SOLR, returns results in expected JSON format
Scrapers ↔ SOLR	Verify scraped data reaches SOLR with correct fields and status
Validator ↔ Core	Verify admin.peviitor.ro status changes propagate to SOLR index

2.10 Non-Functional Testing

2.10.1 Performance Testing

- **Search Response Time**: P95 < 2s for standard queries, P99 < 5s for complex faceted queries
- **Indexing Throughput**: 40,000+ jobs indexed/updated daily within batch window
- **Concurrent Users**: Support 50+ simultaneous users on Raspberry Pi infrastructure
- **API Latency**: P95 < 500ms for API-only endpoints (excluding network)

- **Frequency**: Quarterly and before major releases

2.10.2 Security Testing

- **OWASP Top 10**: Verify application against the OWASP Top 10 web application security risks (broken access control, XSS, SQL injection, etc.)
- **SOLR Authentication**: Verify basic auth is enabled on solr.peviitor.ro (see `008_enable_solr_basic_auth_pi.sh`)
- **API Input Sanitization**: Verify SQL injection / XSS attempts are rejected
- **CORS Headers**: Verify proper CORS configuration on API
- **Sentry Error Monitoring**: No sensitive data leaked in error logs
- **GitHub Secret Scanning**: Automatically detects accidental commits of credentials, API keys, tokens, and other secrets across all repositories in the peviitor-ro organization. Alerts are sent to repository admins.
- **GitHub Code Scanning (CWE)**: Uses CodeQL to analyze code for security vulnerabilities mapped to Common Weakness Enumeration (CWE). Runs on every push and pull request. Results appear in the Security tab of each repository.
- **Dependabot**: Automated dependency monitoring that checks all libraries and packages for known vulnerabilities. Creates pull requests to update vulnerable dependencies to the latest patched version. Configured via `dependabot.yml` in each repository.
- **Frequency**: Bi-annually and after infrastructure changes; automated scanning runs on every push/PR via GitHub

2.10.3 Accessibility Testing

- **WCAG Compliance**: Target WCAG 2.2 Level AA — the current W3C Recommendation (October 2023) and the most widely adopted legal standard globally
- **Keyboard Navigation**: All functionality accessible via keyboard (Tab, Enter, Escape, arrow keys)
- **Screen Reader**: Proper ARIA labels, semantic HTML, alt text on images
- **Color Contrast**: Minimum 4.5:1 contrast ratio for normal text
- **Frequency**: Per release and during UI changes

2.10.4 Usability Testing

- **Search Flow**: Intuitive search, clear filters, easy job detail access
- **Mobile Experience**: Touch-friendly, readable on small screens, fast mobile load
- **Onboarding**: First-time user understands how to search and filter within 30s
- **Error Messages**: Clear, helpful error messages (e.g., "No jobs found" with suggestions)
- **Frequency**: Per release and after UX/UI changes

2.10.5 Reliability / Availability

- **Target**: Platform uptime 99.5% (excluding planned maintenance)
- **Monitoring**: Sentry + CloudFlare analytics

2.10.6 Data Accuracy

- **Target**: Maximum 1% stale/incorrect jobs in index
- **Approach**: URL validation pipeline runs daily to detect and remove dead links

2.10.7 Disaster Recovery

- **Scope**: SOLR index backup and restoration

- **Documentation**: Restore procedure documented in peviitor_core
- **Frequency**: Daily automated backups

2.10.8 Browser Compatibility

- **Scope**: Latest 2 versions of Chrome, Firefox, Edge, Safari
- **Note**: Defined as a prerequisite before development — not a runtime test

2.10.9 Mobile Responsiveness

- **Scope**: All pages functional on viewports ≥ 320px width
- **Approach**: Test on real devices and Chrome DevTools device emulation

2.10.10 SEO

- **Meta Tags**: Verify presence and correctness of title, description, Open Graph tags
- **Semantic HTML**: Proper heading hierarchy (h1–h6), landmark elements
- **Tools**: Lighthouse SEO audit, manual review

2.10.11 Cross-Browser Testing

- **Scope**: Validate UI rendering and functionality across Chrome, Firefox, Edge, Safari — latest 2 versions each
- **Approach**: Manual functional testing on each browser; verify layout, functionality, and error states are consistent
- **Responsive Design**: Test on viewport widths from 320px to 1920px across all browsers
- **Tools**: BrowserStack (cross-browser cloud), Chrome DevTools device emulation, manual with real devices
- **Frequency**: Per release and after major UI changes

---

3. TEST TEAM

The active team is listed at oportunitatisicariere.ro/echipa.html. Because the project relies on volunteers, the team is constantly changing — this document captures the current roles and responsibilities rather than fixed names.

3.1 Team Structure

Role	Number of Testers	Responsibilities
Manual QA Tester	5	Functional testing, regression testing, exploratory testing, bug reporting, test case creation, cross-browser testing, data validation via admin.peviitor.ro
Automation QA Tester	2	Automated E2E tests (Playwright), API test automation (Postman collections), CI/CD integration, test script maintenance, regression automation
Performance Tester	1	Load testing (k6), SOLR query performance monitoring, API latency measurement, Lighthouse audits, capacity planning
Security Tester	1	OWASP Top 10 validation, penetration testing (OWASP ZAP), vulnerability assessment, security code review, Dependabot alert triage, secret scanning monitoring

3.2 Responsibilities per Role

Manual QA Testers (5)

- Execute test cases for each release
- Perform exploratory testing on new features
- Log and track bugs in GitHub Issues
- Verify fixed bugs and close them
- Maintain test case repository
- Run regression checklists before production deploy
- Validate data quality via admin.peviitor.ro

Automation QA Testers (2)

- Develop and maintain automated test suites (Playwright, API)
- Integrate tests into GitHub Actions CI/CD pipeline
- Review manual test cases for automation potential
- Maintain test data and fixtures
- Report automation coverage metrics

Performance Tester (1)

- Conduct load tests on search and API endpoints
- Monitor SOLR query performance and indexing times
- Track Lighthouse scores per release
- Set up performance dashboards
- Identify bottlenecks and recommend optimizations

Security Tester (1)

- Perform security assessments against OWASP Top 10
- Configure and monitor GitHub Secret Scanning, Code Scanning (CodeQL), and Dependabot
- Conduct manual security reviews of API endpoints
- Verify CORS, authentication, and input sanitization
- Generate security test reports

3.3 Training & Onboarding

New testers will be onboarded via the peviitor.ro onboarding portal and receive access to the tools listed in section 2.3 (TOOLS). Each tester will be assigned a mentor from the existing team for the first 2 weeks.

---

4. TEST LEVELS / TESTING TYPES

4.1 Unit Testing

Layer	Scope	Responsibility
PHP (api / peviitor_core)	Utility functions, data transformations, SOLR query builders	Development team
JavaScript (search-engine)	Redux slices, selectors, utility functions	Frontend team
Tools	PHPUnit, Jest, Vitest	
Frequency	On each PR / commit	

4.2 Integration Testing

Integration	Scope
API ↔ SOLR	Verify API queries return correct SOLR results
Frontend ↔ API	Verify network layer, response parsing, error states
Scrapers → Core	Verify scraped data is correctly parsed and indexed
Tools	PHPUnit (integration), Postman collections, manual
Frequency	Weekly, before release

4.3 System Testing (End-to-End)

- **Full search flow**: user lands on peviitor.ro → searches by keyword → filters by location/workmode → views job details → clicks apply link
- **Admin validation flow**: admin.peviitor.ro → review scraped jobs → validate/reject → verify status change in SOLR
- **Cross-browser**: Chrome, Firefox, Edge, Safari
- **Mobile**: responsive layout on 320px–1920px viewports
- **Tools**: Manual exploratory, Playwright
- **Frequency**: Per release

4.4 Regression Testing

- **Critical path regression**: search, filter, pagination, job detail, company listing
- **Scope**: Every release verifies all existing functionality is unaffected
- **Tools**: Manual checklists, Postman collections, automated smoke suite
- **Frequency**: Per release (~weekly)

4.5 Acceptance Testing

- **Internal acceptance**: QA team validates against acceptance criteria from requirements
- **Community feedback**: Bug reports from Discord, GitHub Issues triaged and verified
- **Sign-off**: QA lead confirms all critical and major issues are resolved before production deploy
- **Frequency**: Per release

---

5. ENVIRONMENTS

5.1 LOCAL ENVIRONMENT

Aspect	Detail
Frontend URL	`http://localhost:3000`
API URL	`http://localhost/api` (via local_environment Docker setup)
SOLR	Local SOLR instance via Docker (separate cores: jobs, companies)
Purpose	Feature development, unit testing, isolated debugging
Who Has Access	Developers
Setup	See local_environment repo

Aspect	Detail
Data	Minimal seed data; scraper output can be indexed manually for testing

5.2 TEST ENVIRONMENT

Component	URL	Purpose
Frontend	https://test.peviitor.ro	Integration testing, UAT, regression before release
API (Swagger UI)	https://test.peviitor.ro/swagger-ui	Interactive API documentation with request/response schemas — v0 (dev/test) and v1 (production) only
SOLR	https://testsolr.peviitor.ro	SOLR querying, schema validation, indexing tests, diacritics search verification

Details:
- **Who Has Access**: QA team, developers
- **Data**: A subset of production data (~5,000 jobs) refreshed periodically
- **SOLR Cores**: `jobs`, `companies` — separate from production cores
- **API Versions**: v0 (dev/test) and v1 (production) — v2–v7 are being migrated into v0/v1 and will be removed
- **CI/CD**: GitHub Actions auto-deploys on PR merge
- **Monitoring**: Sentry.io integrated; errors triaged by severity

5.2.1 Build Definition

A **BUILD** is a deployable version of one or more components that has passed CI/CD and is ready for testing on the TEST environment.

Component	What Constitutes a Build	Version Source
Frontend (search-engine)	A merged PR to `main` branch that passes GitHub Actions (lint + build)	Git tag / release (e.g. `v62`)
API (api.peviitor.ro)	A merged PR to `master` branch that passes GitHub Actions	Commit hash + release tag (e.g. `v1.2`)
Core / SOLR config (peviitor_core)	A merged PR to `main` branch with SOLR schema or workflow changes	Commit hash

Build ID format: `[component]-v[number]-[YYYYMMDD]` (e.g. `frontend-v62-20260601`)

A full **release build** includes all three components deployed together with a matching release tag.

5.2.2 Deploy Process to TEST Environment

Step	Who	Action
1	**Developer**	Merges PR to `main` / `master` branch
2	**GitHub Actions**	Auto-triggers CI/CD pipeline: lint → test → build → deploy to test.peviitor.ro
3	**GitHub Actions**	Notifies in Discord `#deploy` channel when deploy completes
4	**QA Tester**	Receives notification and begins smoke testing
5	**QA Tester**	Verifies the build is stable (smoke tests pass)
6	**QA Tester**	Proceeds with full regression / feature testing

Deploy triggers:
- **Automatic**: Every merge to `main` / `master` triggers a deploy to TEST
- **Manual**: A developer or QA lead can trigger a manual re-deploy via GitHub Actions workflow_dispatch

Rollback: If the deployed build is broken, the previous build is re-deployed via GitHub Actions using the last known good commit.

5.3 PRODUCTION ENVIRONMENT

Component	URL	Purpose
Frontend	https://peviitor.ro	Live platform — public facing
SOLR	https://solr.peviitor.ro	Production SOLR querying, schema management, monitoring

Details:
- **Who Has Access**: All users (frontend); QA lead & DevOps (SOLR admin with basic auth)
- **Data**: Full dataset — 40,000+ jobs updated daily via scrapers
- **Deploy**: Manual deploy with QA lead sign-off; GitHub Actions build only
- **Monitoring**: Sentry.io, CloudFlare analytics

5.4 INFRASTRUCTURE

The **Infrastructure Design Document (IDD)** at idd.peviitor.ro is the authoritative reference for all infrastructure details — server specs, network topology, reverse proxy configuration, backup strategy, and deployment architecture. The tables below provide a summary.

5.4.1 Test Infrastructure

Component	Technology	Details
Application Server	Raspberry Pi 4	Hosts Apache, PHP API, NginX Proxy Manager
Frontend Hosting	GitHub Pages	Delivers search-engine UI from `main` branch
SOLR	Test SOLR instance on testsolr.peviitor.ro	Separate cores for jobs & companies; subset of production data
Reverse Proxy	NginX Proxy Manager	Routes traffic, manages SSL, proxies API requests
CDN / DNS	CloudFlare	Caching, DNS, SSL termination
CI/CD	GitHub Actions	Auto-deploys to test environment on PR merge
Monitoring	Sentry	Error tracking for frontend & API

5.4.2 Production Infrastructure

Component	Technology	Details
Application Server	Raspberry Pi 5 (16GB RAM)	Hosts NginX Proxy Manager, Apache, PHP API, and SOLR — single point of failure
Frontend Hosting	GitHub Pages	Delivers search-engine UI to end users
SOLR	SOLR instance on solr.peviitor.ro	Full dataset — 40,000+ jobs updated daily

Component	Technology	Details
Reverse Proxy	NginX Proxy Manager	Routes traffic, manages SSL certificates, proxies API requests to SOLR
CDN / DNS	CloudFlare	Caching, DDoS protection, DNS resolution, SSL termination
Static Sites	Google Sites	Hosts certain project sites
CI/CD	GitHub Actions	Build only — manual deploy with QA lead sign-off
Monitoring	Sentry, CloudFlare analytics	Error tracking, performance analytics

6. DELIVERABLES

Test Artifacts

Artifact	Description	Owner
Test Strategy (this document)	Overall test approach and methodology	QA lead
Test Cases	Detailed step-by-step test cases for each component	QA team
Test Checklists	Quick regression checklists for releases	QA team
Bug Reports	Issues logged in GitHub Issues with severity, steps to reproduce, screenshots	QA team
Test Summary Report	Summary of test execution per release: passed/failed/blocked metrics	QA lead

Reporting Cadence

Report	Frequency	Audience
Bug triage summary	Weekly	QA team, developers
Test execution report	Per release	QA lead, project lead
Release readiness status	Before each production deploy	All teams

Quality Metrics & KPIs

Metric	Target	How It's Measured
Test Pass Rate	≥ 95% of all test cases passing per release	Test execution report
Automation Coverage	≥ 80% critical path coverage	Playwright test results vs. critical path checklist
Defect Escape Rate	≤ 3 high-severity bugs found in production per quarter	Bugs reported on peviitor.ro vs. those caught in testing
Search Response Time	P95 < 2s, P99 < 5s	Lighthouse / k6 performance test results
Accessibility Compliance	Zero critical WCAG violations per release	axe DevTools / Lighthouse audit
Regression Pass Rate	100% pass rate before production deploy	Regression test suite results

7. TEST DATA

This section defines the test data categories used across all testing levels. Test data is sourced from the production index (subset on test environment) and supplemented with synthetic data for edge cases.

7.1 Valid Data

Category	Examples	Notes
Job Titles	"QA", "Java Developer", "software engineer", "accountant"	Existing jobs in the index
City Input	"București", "Cluj", "Iași", "a" (single char minimum)	Accepts at least 1 character; Romanian diacritics supported
Company Names	"P&G", "Bit Sentinel", "EMAG"	At least 3 characters required by UI
Work Mode	remote, on-site, hybrid	Single or multi-select via checkboxes
Combined Filters	City + Work Mode, Company + City, all filters simultaneously	Results must match all selected criteria
Direct Search	"Cluj remote QA", "București software engineer"	User types free-form search with filter values

7.2 Invalid Data

Category	Examples	Expected Behavior
Company < 3 chars	"AB", "X", ""	UI should reject or API return 400; clear error message
Non-existent values	City: "Atlantis", Company: "FakeCorp999", random strings ("xyz123")	Empty results with "no jobs found" message
Special characters	`<script>`, `DROP TABLE`, `%`, `_`, `\`	Input sanitized; no XSS or injection; returned as literal search
Invalid city input	Numbers-only city, symbols in city field	Treated as search term; zero or unexpected results

7.3 Edge Cases

Scenario	Input	Expected Behavior
Very long input	> 500 characters	Truncated or rejected gracefully; no crash
Empty search	Empty string or only whitespace	Either no-op (button disabled) or returns all results
Case sensitivity	"qa" vs "QA", "BUCURESTI" vs "București"	Search is case-insensitive; diacritics-insensitive
Rapid typing	Fast consecutive keystrokes in search input	Debounced API calls; no duplicate requests or race conditions
Overlapping input	Filter values typed directly into search	Correctly interpreted and combined with selected filters
All filters + search	Max combination: query + city + company + multiple work modes	Correct intersection of all criteria

Scenario	Input	Expected Behavior
UI overflow	Long company name, very long job title in card	Text truncated with ellipsis; no layout breakage
Pagination edges	Exactly 10 results, 11 results, 0 results after last page	Correct page count; "no more results" state on last page

7.4 No-Result Scenarios

Scenario	Input	Expected Behavior
Valid but unmatched	"nuclear physicist" (a real job title not in index)	"No jobs found" message with suggestion to broaden search
Over-filtered	City: "București" + Company: "NonExistent SRL"	Empty results; clear message indicating no matches
Expired jobs	Search for a job that has passed its expiration date	Not displayed in results; no error
Invalid URL job	Job whose `url` returns 404	Marked as `tested` (not `published`); excluded from search

7.5 Test Data Sources

Source	Description	Used For
Production SOLR subset	~5,000 jobs copied to test SOLR instance	Functional, integration, regression testing
admin.peviitor.ro	Manual data entry and status transitions	Data validation, status flow testing
Synthetic test data	Manually crafted JSON payloads for edge cases	API testing (boundary, invalid, missing fields)
Scraper output	Freshly scraped jobs from target websites	End-to-end, data accuracy, pipeline testing

---

8. ENTRY CRITERIA

Before testing begins on a new release or feature, the following conditions must be met:

- [ ] Code has been merged to the target branch (main/master)
- [ ] Build passes successfully on CI/CD (GitHub Actions)
- [ ] Frontend builds without errors (`npm run build`)
- [ ] API returns healthy status (smoke test passes)
- [ ] SOLR cores are accessible and contain test data
- [ ] Test environment is deployed with the latest build
- [ ] Test cases / checklists are updated for new features
- [ ] Known critical bugs from previous release are fixed or documented as known issues

---

9. EXIT CRITERIA

A release is considered ready for production when all of the following are met:

- [ ] All S1 (Blocker), S2 (Critical), and S3 (Major) severity bugs are fixed and verified
- [ ] Regression test suite (critical path) passes: 100% pass rate
- [ ] New features are tested and accepted by QA
- [ ] Accessibility checks pass (no critical WCAG violations)
- [ ] Performance benchmarks are within acceptable thresholds (P95 < 2s search)
- [ ] Test summary report is reviewed and approved by QA lead
- [ ] Production deploy receives sign-off from QA lead and project lead

---

10. BUG LIFECYCLE

10.1 Bug Reporting Process

All bugs are tracked in **GitHub Issues** under the relevant repository (search-engine, api, peviitor_core). Each bug report must include:

- **Title**: Clear, descriptive summary of the issue
- **Environment**: test.peviitor.ro / peviitor.ro / local
- **Browser / Device**: Chrome, Firefox, Edge, Safari + version
- **OS**: Windows, macOS, Linux, Android, iOS
- **Steps to Reproduce**: Numbered steps to recreate the bug
- **Expected Result**: What should happen
- **Actual Result**: What actually happens
- **Screenshots / Video**: Visual evidence (if applicable)
- **Severity**: See 10.5

10.2 Defect Triage

All newly reported bugs go through a triage process before being assigned:

Step	Who	Action
1	**QA Tester**	Reports bug with all required fields (see 10.1); adds label `status/triage`
2	**QA Lead**	Reviews daily — confirms reproduction, checks for duplicates, validates severity
3	**QA Lead**	Removes `status/triage` label; adds `bug` label; assigns to developer or rejects with reason
4	**Project Manager**	Sets priority based on business impact (see 10.4)
5	**Developer**	Picks up assigned bug and moves to In Progress

Triage happens **daily** during weekdays. Critical (S2) and Blocker (S1) bugs are triaged within **4 hours** of reporting.

10.3 Bug States

- **Severity**: See 10.5

```
[New] → [Assigned] → [In Progress] → [Fixed] → [Verified] → [Closed]
                  ↘ [Rejected] → [Closed]
[Verified] → [Reopened] → [Assigned]
```

State	Description
New	Bug reported, not yet reviewed
Assigned	Assigned to a developer for fixing
In Progress	Developer is actively working on the fix
Fixed	Fix is deployed to test environment
Verified	QA has tested and confirmed the fix
Closed	Bug is resolved and accepted
Rejected	Not a bug / duplicates / cannot reproduce
Reopened	Bug reappears after verification

10.4 Priority

Priority describes the **business urgency** of fixing the bug. It is set by the **Project Manager** during triage and may change based on release deadlines or business impact.

Level	Label	Description	Typical SLA
P1 - Critical	`priority/critical`	Must be fixed immediately — blocks the release	Fixed within 24h
P2 - High	`priority/high`	Should be fixed before the next release	Fixed within current sprint
P3 - Medium	`priority/medium`	Fix in a future release if time permits	Next sprint or later
P4 - Low	`priority/low`	Fix when convenient — nice to have	No fixed deadline
P5 - Wishlist	`priority/wishlist`	Tracked for future consideration	No deadline

10.5 Severity

Severity describes the **technical impact** of the bug (how bad the problem is). Priority is set separately by the Product Owner based on business urgency and is tracked via the labels above.

Severity Levels

Level	Label	Description	Example
S1 - Blocker	`severity/blocker`	The application or a critical component is completely unusable. No workaround exists. Testing cannot proceed until resolved.	Search returns HTTP 500; SOLR core unavailable; page fails to load
S2 - Critical	`severity/critical`	A major feature is broken, data is incorrect or lost, or the user experience is severely degraded. No reasonable workaround exists.	Search returns zero results for valid queries; job details page crashes; incorrect salary data displayed
S3 - Major	`severity/major`	A feature does not work as expected but a partial workaround exists. Functionality is impaired but not completely blocked.	Filters return incorrect results on edge cases; pagination skips pages; search timeout on complex queries
S4 - Minor	`severity/minor`	A feature works correctly but has a minor issue that does not affect functionality. A workaround is easily available.	UI misalignment on tablet; typo in a label; missing tooltip
S5 - Trivial	`severity/trivial`	Cosmetic or low-impact issue. Does not affect functionality or user experience in a	Slightly off color shade; missing alt text on decorative image; extra

Level	Label	Description	Example
		meaningful way.	whitespace

11. RISK AND MITIGATION

Risk	Description	Probability	Impact	Mitigation
Single point of failure — Raspberry Pi 5 (16GB RAM)	Production server runs on a single Raspberry Pi 5 running NginX Proxy Manager, Apache, PHP API, and SOLR. Hardware failure would take down peviitor.ro entirely.	Low	Critical	Daily SOLR backups; documented restore procedure; recovery plan via secondary infrastructure
Data quality from scrapers	Scrapers may return incomplete or incorrect data (missing fields, wrong salary, dead URLs).	Medium	High	Daily URL validation pipeline (HEAD requests, 404 deletion); admin.peviitor.ro manual validation; status flow (scraped → tested → verified → published)
Scraper breakage	Target websites change their HTML structure, CSS selectors, or add anti-bot measures (CAPTCHA, IP blocking).	Medium	Major	Community-driven maintenance; multiple scraper technologies (Python, JS, JMeter); quick fallback to "tested" status
SOLR index corruption	SOLR index may become corrupted due to power failure, disk full, or software bug.	Low	Critical	Automated daily cron jobs for index optimization; backup scripts in peviitor_core; separate test SOLR for validation before production indexing
Volunteer availability	The project relies on volunteers. Key people may become unavailable, causing delays.	Medium	Medium	Cross-training across team members; documentation in GitHub; onboarding process for new volunteers
API rate limiting / blocking	Job portals may block peviitor scrapers or the API may be rate-limited by SOLR or CloudFlare.	Low	Major	Respect robots.txt; configurable scrape intervals; monitoring via Sentry and CloudFlare
GitHub Actions free tier limits	Free tier has monthly usage limits. Heavy CI/CD usage may exhaust the quota.	Low	Minor	Optimize workflows; cache dependencies; remove redundant triggers
CloudFlare CDN outage	CloudFlare is used as CDN and DNS provider. An outage would affect availability.	Very Low	Major	DNS failover not configured; accept as acceptable risk given CloudFlare's 99.99% SLA
Security breach (SOLR / API)	Unauthorized access to SOLR admin or API could leak or corrupt data.	Low	Critical	SOLR basic auth enabled; CORS restrictions; GitHub Secret Scanning; regular dependency updates via Dependabot

12. CONCLUSION

This Test Strategy provides a structured framework for validating the peviitor.ro job search engine across all components — from SOLR indexing and the PHP API to the React frontend and the scraper pipeline.

By defining clear requirements sources, test data categories, entry/exit criteria, and a risk-aware mitigation plan, this document ensures that every release meets the functional, performance, security, accessibility, and usability standards required to serve job seekers across Romania.

The strategy is a living document that evolves with the platform. As new features are added, technologies change, or the community grows, this document will be updated to reflect the current testing approach. All team members are encouraged to propose improvements via GitHub Issues.

Following this strategy, we aim to deliver a reliable, fast, and user-friendly job search experience that fulfills peviitor.ro's primary mission: providing a comprehensive, up-to-date aggregated database of all jobs in Romania.

Appendix A — Test Artifacts

A.1 Bug Report Template

```
 **Title**: [Short descriptive summary]

**Environment**: test.peviitor.ro | peviitor.ro | local
    - **Browser**: Chrome 125
    - **OS**: Windows 11

**Steps to Reproduce**:
1. Go to https://test.peviitor.ro
2. Search for "software engineer"
3. Click "Load More" 3 times
4. Observe results

**Expected Result**: 30 results should be displayed (10 per page × 3 pages).

**Actual Result**: Only 10 results shown after clicking "Load More" 3 times; console shows
404 error.

**Severity**: S2 - Critical

**Screenshots**: [link or attachment]

**Additional Context**: Network tab shows GET /api/v6/search?page=3 returns 404
```

A.2 Test Case Example

```
 Test Case ID: TC-SEARCH-001
Title: Verify basic keyword search returns relevant results
Feature: Search Engine
Environment: test.peviitor.ro
Preconditions: Browser open at https://test.peviitor.ro

Test Steps:
1. Click on the search input field
2. Type "software engineer" and press Enter
3. Wait for results to load
4. Observe the list of displayed job results
```

```
Expected Result:
- Results are displayed within 2 seconds
- Each result contains: job title, company name, location
- Results contain the keyword "software" or "engineer" or both
- Pagination controls are visible (if more than 10 results)
- No error messages are shown
```

A.3 Test Execution Report Template

```
 TEST EXECUTION REPORT

Release: [vXX]
Date: [DD/MM/2026]
Tester: [Name]
Environment: test.peviitor.ro

1. EXECUTION SUMMARY
   Total Test Cases:      XX
   Passed:                XX (XX%)
   Failed:                XX (XX%)
   Blocked:               XX (XX%)
   Not Executed:          XX (XX%)

2. BUG SUMMARY
   S1 - Blocker:          XX
   S2 - Critical:         XX
   S3 - Major:            XX
   S4 - Minor:            XX
   S5 - Trivial:          XX
   Total Bugs Found:      XX

3. TEST COVERAGE
   Frontend:              XX/XX (XX%)
   API:                   XX/XX (XX%)
   SOLR Indexing:         XX/XX (XX%)
   Data Validation:       XX/XX (XX%)
   Cross-Component:       XX/XX (XX%)

4. KEY FINDINGS
   - [Critical issue found in ...]
   - [Major regression in ...]
   - [All accessibility checks passed]

5. RECOMMENDATION
   [Approve / Conditional Approve / Reject] for production deployment.

6. SIGN-OFF
   Tester: _____________________
   QA Lead: _____________________
   Date: ___/___/2026
```

Appendix B — GitHub Repositories

Component	Repository	URL
Frontend (current)	search-engine	github.com/peviitor-ro/search-engine
Frontend v01	v01	github.com/peviitor-ro/v01
API	api.peviitor.ro	github.com/peviitor-ro/api.peviitor.ro
Core (indexing, schemas, workflows)	peviitor_core	github.com/peviitor-ro/peviitor_core
Validator UI	validator-ui	github.com/peviitor-ro/validator-ui
Validator (legacy)	admin.peviitor.ro	github.com/peviitor-ro/admin.peviitor.ro
Mobile app (separate strategy)	androidAPP	github.com/peviitor-ro/androidAPP
Scrapers frontend	frontend	github.com/peviitor-ro/frontend
Scrapers scraping	scraping	github.com/peviitor-ro/scraping
Local dev environment	local_environment	github.com/peviitor-ro/local_environment
Landing page	landing-page	github.com/peviitor-ro/landing-page
Documentation	documentation	github.com/peviitor-ro/documentation
DevOps / hosting	devops	github.com/peviitor-ro/devops

Appendix C — Data Models

C.1 Job Model

Field	Type	Required	Description & Rules
`url`	string	Yes	Full URL to the job detail page. Must be unique, valid HTTP/HTTPS, canonical job detail page
`title`	string	Yes	Exact position title. Max 200 chars, no HTML, trimmed whitespace, diacritics accepted (ăâîșț)
`company`	string	No	Legal name of the hiring company. Must match Company.name (case insensitive, diacritics accepted)
`cif`	string	No	CIF/CUI of the company
`location`	string[]	No	Romanian cities/addresses. Diacritics accepted. Multi-valued array
`tags`	string[]	No	Skills/education/experience. Lowercase, max 20 entries, no diacritics
`workmode`	string	No	One of: "remote", "on-site", "hybrid"
`date`	date	No	Scrape/index date (ISO8601 UTC)
`status`	string	No	One of: "scraped", "tested", "published", "verified"
`vdate`	date	No	Verified date (ISO8601). Set only when validation is "verified"
`expirationdate`	date	No	Estimated expiration date = vdate + 30 days max
`salary`	string	No	Salary interval + currency (e.g. "5000-8000 RON"). Must be a string

Job Status Flow: `scraped` → (`tested` OR `verified`) → `published`

C.2 Company Model

Field	Type	Required	Description & Rules
`id`	string	Yes	CIF/CUI (e.g. "12345678"). 8 digits, no RO prefix
`company`	string	Yes	Legal name from Trade Register. Uppercase, diacritics required
`brand`	string	No	Commercial brand name (e.g. "ORANGE", "EPAM")
`group`	string	No	Parent company group
`status`	string	No	One of: "activ", "suspendat", "inactiv", "radiat"
`location`	string[]	No	Romanian cities/addresses. Diacritics accepted. Multi-valued
`website`	string[]	No	Official company website(s). Valid HTTP/HTTPS URLs. Multi-valued
`career`	string[]	No	Company career page(s). Valid HTTP/HTTPS URLs. Multi-valued
`lastScraped`	string	No	Date of last scrape (ISO8601)
`scraperFile`	string	No	Name of the scraper file used

Appendix D — Domain & Infrastructure Configuration

Item	Configuration
Cloudflare account email	`sebitestb@gmail.com`
DNS records	All DNS records are managed in Cloudflare